

Bricks: A Habit-Building Wall Progression App

Initial Project Specification + Technical Planning Bundle

1. Project Overview

Bricks is a local Android habit-building app designed to help a user train a habit through a fixed visual progression system. The app turns completed habit days into bricks in a wall. The purpose is not to become a permanent productivity crutch, but to act like **training wheels** until the user has repeated the habit long enough to internalize it.

The core reward is visual: each completed scheduled day lays one brick. Over time, the user sees a wall being built. If the user uploads an image for the habit, the image is gradually revealed through the brick wall. When the current tier is completed, the full image becomes clear.

The app is being built by the user for personal use, so the MVP should avoid unnecessary account systems, cloud sync, social features, onboarding, or overbuilt customization.

2. Product Vision

Bricks should feel strict, simple, visual, and personal.

The user should not be rewarded for partial effort. A brick is earned only when the complete scheduled habit task list is finished within the required time window.

The app should create a sense of progress through:

- visible wall construction,
- tiered habit strengthening,
- strict missed-day consequences,
- daily brick-laying animation,
- optional image selection,

- and a clear end-state for each tier.

The long-term identity of the app is:

Build the habit. Fortify it. Eventually outgrow the app.

3. Core Psychological Loop

The user feels compelled to see their progress visualized.

They open Bricks, choose the habit they want to work on, and complete each required task by checking it off. When all tasks are completed, an animation plays showing a brick being laid into the wall.

The user returns because they want to see the next brick added the next scheduled day. Over time, the reward becomes the completed wall itself, especially if the user uploaded an image that slowly becomes clearer as the habit develops.

The loop is:

1. User wants visible progress.
 2. User opens Bricks.
 3. User selects a habit.
 4. User completes all required tasks.
 5. App lays one brick.
 6. User sees the wall grow.
 7. User returns to continue the wall.
-

4. MVP Scope

The MVP should include:

- local-only Android app;
- Kotlin;
- Jetpack Compose UI;
- single-activity architecture;
- Room local database;
- daily and specific-weekday habit schedules;
- one reminder time per habit;
- active habit library view;
- centered habit view;

- habit creation;
 - habit deletion with confirmation;
 - no habit editing after creation;
 - task checklist;
 - strict completion locking;
 - brick wall rendering;
 - optional uploaded image mapped across the wall;
 - Bronze/Silver/Gold progression;
 - missed-day rules;
 - midnight rollover processing;
 - notification for scheduled habit reminders;
 - notification for missed scheduled intervals;
 - modular project structure prepared for future widget support.
-

5. Out of Scope for MVP / Future Enhancements

The following should not be implemented in the initial version, but should be preserved as future ideas:

- home-screen widget;
 - failed/abandoned wall history;
 - custom interval habits;
 - custom wall goal lengths;
 - multiple reminders per habit;
 - cloud sync;
 - account login;
 - export/backup;
 - social sharing;
 - onboarding/tutorial screens;
 - theme customization;
 - analytics dashboard;
 - streak charts;
 - habit editing;
 - task editing;
 - advanced image effects;
 - custom tier naming;
 - habit templates.
-

6. Core Habit Rules

6.1 Habit Creation

A habit is created with:

- habit name;
- schedule type:
 - daily, or
 - specific weekdays;
- one reminder time;
- one shared task list;
- optional uploaded image.

A habit must have at least one task.

A habit name must be unique among active habits.

Duplicate active habit names are not allowed.

Deleted habit names may be reused later.

6.2 Habit Editing

For MVP, habits are **not editable**.

This includes:

- habit name;
- task list;
- schedule;
- reminder time;
- uploaded image;
- tier goal.

If the user wants to change a habit, they must delete it and create a new habit.

This keeps the habit definition stable and avoids complicated state migration.

6.3 Habit Deletion

Deleting a habit requires a confirmation dialog.

The confirmation should clearly warn that deleting the habit permanently removes:

- habit name;
- task list;
- schedule;
- reminder;
- progress;
- wall state;
- uploaded image.

No undo or recovery is included in MVP.

7. Brick-Earning Rule

A brick is earned only when **all tasks inside the habit are completed within that habit's scheduled day**.

There is no partial credit.

There are no optional tasks.

There is no percentage-based completion.

There is no manual “mark day complete” override.

Official rule:

One scheduled day + all tasks completed within that day = one earned brick.

8. Time Window Rule

A scheduled day is a strict local 24-hour calendar window.

For MVP:

- day starts at **12:00 AM local device time**;
 - day ends at **11:59:59 PM local device time**;
 - after midnight, incomplete scheduled tasks are considered missed;
 - the user cannot complete yesterday's habit after midnight;
 - this applies to both daily habits and specific-weekday habits.
-

9. Tier Progression System

Bricks uses a built-in tier system.

There are no custom tier lengths in MVP.

9.1 Tiers

Tier	Visual Label	Border	Total Wall Size
Bronze	Habit	Bronze border	30 bricks
Silver	Routine	Silver border	60 bricks
Gold	LifeStyle	Gold border	90 bricks

The backend object remains a **Habit** regardless of tier.

The labels **Habit**, **Routine**, and **LifeStyle** are frontend visual interpretations based on progress.

9.2 Fortify Behavior

When the user completes the Bronze wall, the wall receives a bronze border and the bottom label says **Habit**.

The user may press **Fortify**.

When Fortify is selected:

- the same habit continues;
- the same task list is used;
- the wall expands from 30 to 60 bricks;
- the previous 30 completed bricks remain laid;
- the user must complete another 30 scheduled days to complete the Silver tier.

When the Silver wall is completed:

- it receives a silver border;
- the bottom label says **Routine**;
- the user may Fortify again.

When Fortify is selected again:

- wall expands from 60 to 90 bricks;
- previous 60 completed bricks remain laid;
- the user must complete another 30 scheduled days.

When the Gold wall is completed:

- it receives a gold border;
 - the bottom label says **LifeStyle**.
-

10. Missed-Day Rules

Missed-day behavior depends on the current tier and schedule type.

Important terminology:

Because MVP supports daily and specific-weekday habits, “missed day” means a missed **scheduled day**.

10.1 Daily Habit: Bronze / Habit Tier

During the initial 30-day Bronze tier:

- user may miss up to **2 consecutive scheduled days**;
- missed days create visible gaps/empty brick slots in the wall;
- future completed bricks continue being laid in their correct positions;
- once the wall canvas is otherwise full, the user may repair the gaps by completing extra scheduled days;
- if the user misses another scheduled day before the gaps are repaired and the Bronze wall is completed, the Bronze tier resets.

Reset means:

- active wall disappears;
- progress restarts clean;

- failed wall history is not preserved in MVP.
-

10.2 Daily Habit: Silver / Routine Tier

During the Silver tier:

- the user may miss up to **1 scheduled day**;
 - the miss creates a visible gap;
 - the user must eventually repair the gap;
 - if the user exceeds the allowed miss threshold of 1 day, the Silver tier resets;
 - After missing the one day, if the user misses **3 consecutive scheduled days**, the habit reverts to the beginning of the Bronze tier.
-

10.3 Daily Habit: Gold / LifeStyle Tier

During the Gold tier:

- any missed scheduled day causes the habit to revert to the beginning of the Silver tier.
-

10.4 Specific-Weekday Habits

For specific-weekday habits, there is **no missed-day allowance**.

Example:

If a habit is scheduled for Monday, Wednesday, and Friday:

- Monday is scheduled;
- Tuesday does not matter;
- Wednesday is scheduled;
- Friday is scheduled.

If the user misses one scheduled weekday, the relevant tier reset/reversion rule triggers immediately.

Non-scheduled days do not count as misses.

11. Wall Rendering Rules

11.1 Wall Growth

Bricks uses one wall that grows across tiers:

- Bronze: 30-brick wall;
- Silver: 60-brick wall;
- Gold: 90-brick wall.

When moving to a higher tier, the wall is re-rendered at the new capacity.

The existing completed bricks remain visible, but the wall layout changes so more bricks fit.

When regressing to a lower tier, the wall is re-rendered with larger bricks.

If an image was selected by the user, each brick holds a part of the entire image and is laid accordingly(in order), so when the current tier is completed the entire image is now visible, when the habit is further fortified the image is re rendered to be incomplete with the previously tier's laid bricks, and vice versa of the tier level goes down.

11.2 Brick Placement Direction

The brick-laying animation should mimic a real wall being built.

Bricks are laid:

- right to left across a row;
- then the next row begins;
- again right to left;
- continuing until the wall is complete.

11.3 Default Brick Appearance

If no image is uploaded, the wall uses default red bricks.

Default brick color:

- red.

Tier border colors:

- Bronze border for Habit;
 - Silver border for Routine;
 - Gold border for LifeStyle.
-

11.4 Uploaded Image Behavior

Image upload is optional.

If the user uploads an image:

- the image is treated as one full image spread across the entire wall;
- each brick displays its corresponding fragment of the image;
- the image is faded while the wall is incomplete;
- when the current tier is complete, the image becomes clear;
- brick separation rules still apply based on tier.

The image is dynamically resized/re-sliced when the wall changes tier size.

The image is not permanently baked into each brick. It should be mapped across the current wall layout at render time.

In other words: If an image was selected by the user, each brick holds a part of the entire image and is laid accordingly(in order), so when the current tier is completed the entire image is now visible, when the habit is further fortified the image is re rendered to be incomplete with the previously tier's laid bricks, and vice versa of the tier level goes down.

11.5 Brick Divider Visibility

Brick divider visibility depends on tier:

Tier	Divider Visibility
Bronze	clearly visible
Silver	half visible

Gold	invisible
------	-----------

In Gold, the wall should feel more like a complete image/art piece.

12. Animation Rules

When the user completes all tasks for a scheduled day:

1. completion is locked for the day;
2. one brick is earned;
3. brick-laying animation starts;
4. wall view appears;
5. completed wall state remains visible.

Animation plays **once per completed scheduled day**.

If the user reopens the habit later that same day, the app shows the completed wall state without replaying the animation.

13. Completion Locking

Once all tasks are checked for the scheduled day:

- the day is marked complete;
- the earned brick is saved;
- the animation begins;
- task completion is locked;
- the user cannot uncheck tasks for that day.

This avoids ambiguous state where a brick has already been earned but the checklist is later modified.

14. Reminder and Notification Rules

14.1 Habit Reminder

Notifications are included in MVP.

When creating a habit, the user must choose one reminder time.

Each habit has one reminder time only.

The reminder fires only on scheduled days.

The reminder notification should reference the habit name.

Example:

Time to build: Workout

If the habit is already completed for the day, the notification should not fire or should be ignored.

14.2 Midnight Miss Processing Notification

At **12:00 AM local device time**, Bricks should evaluate scheduled habits from the previous day.

If a scheduled habit was incomplete, Bricks should:

1. process the miss;
2. apply the correct tier policy;
3. update the wall state;
4. send a notification explaining what happened.

Example Bronze message:

You missed Workout. A gap was added to your Habit wall. Complete an extra scheduled day later to repair it.

Example Gold message:

You missed Workout. Your LifeStyle wall reverted to Routine because Gold allows no missed days.

Miss detection should also run when the app launches, in case midnight processing was delayed because the device was off or background execution was restricted.

The app must store enough state to avoid processing the same missed day more than once.

15. UI / UX Specification

15.1 App Launch

The app opens directly to **Library View**.

No onboarding screen.

No tutorial.

No login.

No account prompt.

15.2 Library View

Library View is the default landing screen.

It displays all active habits as cards.

Target layout:

- 3 cards across;
- 5 cards vertically visible;
- approximately 15 visible habits per screen;
- scrolling if more habits exist.

Each habit card should display:

- habit name;
 - current tier:
 - Habit / Bronze,
 - Routine / Silver,
 - LifeStyle / Gold;
 - today's completion status;
 - number of bricks laid;
 - faded image preview if image exists;
 - red brick preview if no image exists.
-

15.3 Habit View

When the user selects a habit card:

- Habit View opens centered;
 - it occupies approximately 80% of the screen;
 - the remaining 20% border still shows the Library View behind/around it;
 - tapping the visible 20% outer area closes Habit View;
 - swiping left/right moves between active habits in library sequence.
-

15.4 Habit View State: Before Completion

Before all tasks are checked, Habit View shows only the task checklist.

The wall is not the primary focus before task completion.

This keeps the user focused on execution first.

15.5 Habit View State: After Completion

Once all tasks are checked:

- completion locks;
- animation plays;
- wall view appears;
- wall remains visible.

In wall view:

- a small three-dot button appears in the top-left;
 - tapping it reveals the task list;
 - the task list appears over a slightly translucent background for contrast.
-

16. Create Habit Flow

MVP creation flow:

1. Enter habit name.

2. Choose schedule:
 - daily, or
 - specific weekdays.
3. Choose one reminder time.
4. Add task names into one shared task list.
5. Optionally upload image.
6. Create habit.

Validation:

- habit name required;
- habit name must be unique among active habits;
- at least one task required;
- reminder time required;
- schedule required.

No editing after creation.

Technical Planning Bundle

17. Technical Stack

Bricks will use:

- **Android Studio**
- **Kotlin**
- **Jetpack Compose**
- **Single-Activity Architecture**
- **Room**
- **WorkManager and/or AlarmManager**
- **Local file storage for uploaded images**

No backend.

No cloud sync.

No account system.

18. Architecture Principles

The main architectural goal is modularity.

The app should avoid becoming a tangled monolith.

Future features should be addable as mostly self-contained packages/modules, with limited changes to existing code.

Preferred structure:

- feature-oriented organization;
 - shared core models;
 - clear state management;
 - repositories for data access;
 - use cases/services for business rules;
 - composables for UI;
 - ViewModels for screen state;
 - no business rules buried inside UI composables.
-

19. Proposed Package Structure

Suggested package root:

com.bricks.app

Suggested structure:

com.bricks.app

|

|— MainActivity.kt

|

|— core

|

| |— model

|

| |— time

|

| |— ui


```
| |— navigation
| |— util
|
|— data
| |— db
| |— dao
| |— entity
| |— mapper
| |— repository
|
|— features
| |— library
| | |— LibraryScreen.kt
| | |— LibraryViewModel.kt
| | |— components
| |
| |— habit
| | |— HabitViewScreen.kt
| | |— HabitViewModel.kt
| | |— components
| |
| |— creation
| | |— CreateHabitScreen.kt
| | |— CreateHabitViewModel.kt
```

```

| | └─ components
| |
| | └─ wall
| |   └─ WallRenderer.kt
| |   └─ WallCanvas.kt
| |   └─ BrickLayoutCalculator.kt
| |   └─ ImageWallMapper.kt
| | └─ animation
| |
| | └─ notifications
| |   └─ ReminderScheduler.kt
| |   └─ MissedDayScheduler.kt
| |   └─ NotificationBuilder.kt
| | └─ workers
| |
| | └─ rollover
| |   └─ DailyRolloverProcessor.kt
| |   └─ MissedDayPolicyEngine.kt
| |   └─ TierTransitionEngine.kt
|
└─ future
    └─ widget

```

The `future/widget` package does not need real implementation yet. It can be left empty physically at first, but the architecture should avoid blocking future widget support.

20. Recommended Core Domain Models

These are conceptual models, not final Kotlin files yet.

Habit

Represents the stable habit definition.

Fields:

- habitId;
 - name;
 - scheduleType;
 - selectedWeekdays;
 - reminderTime;
 - imageUri/path nullable;
 - createdAt;
 - isActive.
-

HabitTask

Represents one task inside a habit.

Fields:

- taskId;
- habitId;
- taskName (basically the string description of the actual task);
- sortOrder.

Tasks are not editable in MVP.

HabitProgress

Represents current progress state.

Fields:

- progressId;
- habitId;
- currentTier;

- totalBricksRequired;
 - completedBrickCount;
 - missedGapCount;
 - currentWallStartDate;
 - lastProcessedDate;
 - lastCompletedDate;
 - tierStatus.
-

HabitDayRecord

Represents the state of one scheduled day.

Fields:

- recordId;
 - habitId;
 - date;
 - wasScheduled;
 - wasCompleted;
 - wasMissed;
 - wasProcessed;
 - brickIndex nullable;
 - isGap;
 - createdAt.
-

TaskCompletionRecord

Represents task-level completion for a specific date.

Fields:

- completionId;
 - habitId;
 - taskId;
 - date;
 - isCompleted;
 - completedAt nullable.
-

21. Proposed Room Entity Outline

Likely Room entities:

HabitEntity

HabitTaskEntity

HabitProgressEntity

HabitDayRecordEntity

TaskCompletionEntity

Possible enum-like fields:

ScheduleType

TierType

DayStatus

Suggested enum values:

ScheduleType:

- DAILY

- SPECIFIC_WEEKDAYS

TierType:

- BRONZE

- SILVER

- GOLD

DayStatus:

- NOT_SCHEDULED

- SCHEDULED_PENDING

- COMPLETED

- MISSED
 - GAP_REPAIRED
-

22. Core State Model

A habit's state is determined by:

1. its schedule;
2. current date;
3. completed task records for today;
4. current tier;
5. completed brick count;
6. missed gap state;
7. last processed date;
8. whether today's scheduled interval has already been completed;
9. whether animation has already played for today.

The UI should not independently decide business rules.

Business logic should live in policy/service classes such as:

MissedDayPolicyEngine

TierTransitionEngine

DailyRolloverProcessor

BrickProgressCalculator

23. Main User Flows

23.1 Create Habit Flow

1. User opens app.
2. User enters Library View.
3. User taps Create Habit.
4. User enters habit name.
5. User selects schedule.
6. User selects reminder time.
7. User adds one or more tasks.

8. User optionally uploads image.
 9. User creates habit.
 10. Habit appears in Library View.
-

23.2 Complete Scheduled Habit Flow

1. User opens Bricks.
 2. User selects habit from Library View.
 3. Habit View opens.
 4. User sees task checklist.
 5. User checks off every task.
 6. App locks completion.
 7. App records completed day.
 8. App increments brick count.
 9. App plays brick-laying animation once.
 10. App shows updated wall.
-

23.3 Missed Day Flow

1. Scheduled day ends.
 2. At midnight, app evaluates previous day.
 3. If incomplete, app marks day missed.
 4. Missed-day policy is applied.
 5. Wall state updates.
 6. Notification is sent.
 7. App prevents double-processing of the same missed day.
-

23.4 Fortify Flow

1. User completes current tier wall.
 2. Wall receives current tier border/label.
 3. Fortify button appears.
 4. User taps Fortify.
 5. Wall expands to next tier size.
 6. Previously completed bricks remain visible.
 7. User continues building.
-

23.5 Delete Habit Flow

1. User selects delete action.
 2. Confirmation dialog appears.
 3. Dialog warns deletion is permanent.
 4. User confirms.
 5. Habit and related records are removed/deactivated.
 6. Habit no longer appears in Library View.
-

24. UML / Diagram Recommendations

Yes, this project should have planning diagrams.

Recommended artifacts:

24.1 Use Case Diagram

Actors:

- User
- Android System Scheduler / Notification System

Use cases:

- Create habit
 - View habit library
 - Open habit
 - Complete tasks
 - Earn brick
 - View wall
 - Fortify habit
 - Delete habit
 - Receive reminder
 - Receive missed-day notification
-

24.2 Class Diagram / UML

Should include:

- Habit
- HabitTask

- HabitProgress
- HabitDayRecord
- TaskCompletionRecord
- HabitRepository
- ReminderScheduler
- DailyRolloverProcessor
- MissedDayPolicyEngine
- TierTransitionEngine
- WallRenderer
- ImageWallMapper

This will help prevent business logic from leaking into UI files.

24.3 ERD / Database Schema Diagram

Should include Room tables and relationships:

- HabitEntity 1-to-many HabitTaskEntity
 - HabitEntity 1-to-one or 1-to-many HabitProgressEntity
 - HabitEntity 1-to-many HabitDayRecordEntity
 - HabitTaskEntity 1-to-many TaskCompletionEntity
-

24.4 User Flow Diagram

Should show:

- app launch;
 - Library View;
 - Create Habit;
 - Habit View;
 - task completion;
 - wall animation;
 - fortify;
 - delete;
 - missed-day processing.
-

24.5 State Machine Diagram

Highly recommended for this app.

Habit state transitions are strict and tier-dependent.

States might include:

- Active Bronze
- Bronze Completed
- Active Silver
- Silver Completed
- Active Gold
- Gold Completed
- Reset Bronze
- Reverted Silver
- Deleted

This will be especially useful for missed-day behavior.

25. Non-Functional Requirements

25.1 Local-Only

All data must remain on-device.

No user login.

No remote backend.

No cloud sync.

25.2 Reliability

The app must correctly process missed days even if:

- the app was not opened yesterday;
- the phone was off at midnight;
- Android delayed background work;
- reminders were not delivered exactly on time.

This is why app-launch reconciliation is required.

25.3 Modularity

The codebase should be structured so future features can be added without major rewrites.

Particularly important future features:

- widget support;
 - failed wall history;
 - custom intervals;
 - export/backup;
 - richer wall rendering.
-

25.4 Simplicity

The MVP should avoid:

- accounts;
 - cloud features;
 - social features;
 - excessive customization;
 - analytics dashboards;
 - habit editing;
 - complex scheduling.
-

26. MVP Development Phases

Phase 1: Project Foundation

Build:

- UML- and Diagrams - Initial
- Android Studio project;
- Kotlin setup;
- Jetpack Compose;
- Room database;
- package structure;
- basic navigation state;
- dark visual foundation.

Goal:

App opens to an empty Library View.

Phase 2: Habit Creation + Local Persistence

Build:

- Create Habit screen;
- habit name validation;
- schedule selection;
- reminder time picker;
- task list builder;
- optional image selection;
- Room save logic.

Goal:

User can create a habit and see it in Library View.

Phase 3: Library View

Build:

- 3x5 card grid;
- habit cards;
- tier display;
- brick count display;
- today completion status;
- faded image/default red preview.

Goal:

User can view all active habits clearly.

Phase 4: Habit View + Task Completion

Build:

- centered 80% Habit View;
- 20% outer close area;
- swipe left/right between habits;

- checklist-only pre-completion state;
- task checking;
- completion locking.

Goal:

User can complete all tasks for a habit day.

Phase 5: Wall Rendering

Build:

- wall canvas;
- brick layout calculator;
- right-to-left brick placement;
- 30/60/90 wall sizes;
- divider visibility by tier;
- default red brick wall.

Goal:

Completed habit days visually become bricks.

Phase 6: Image Mapping

Build:

- image storage;
- image-to-wall mapping;
- faded image fragments;
- clear image on tier completion;
- dynamic resizing across 30/60/90 wall layouts.

Goal:

Uploaded images reveal progressively through the wall.

Phase 7: Animation

Build:

- brick-laying animation;
- animation plays once per completed scheduled day;
- wall remains after animation.

Goal:

Completing a habit visibly lays a brick.

Phase 8: Tier Progression + Fortify

Build:

- Bronze completion;
- Fortify button;
- Silver expansion;
- Gold expansion;
- tier labels;
- tier borders.

Goal:

Habit can progress from Habit → Routine → LifeStyle.

Phase 9: Missed-Day Policy Engine

Build:

- daily rollover processor;
- missed-day detection;
- gap creation;
- reset behavior;
- revert behavior;
- specific-weekday no-allowance rule;
- duplicate-processing prevention.

Goal:

App correctly enforces habit discipline.

Phase 10: Notifications

Build:

- habit reminder notifications;
- midnight missed-day notifications;
- launch reconciliation.

Goal:

User receives reminders and miss-policy notifications.

Phase 11: Delete Habit

Build:

- delete action;
- confirmation dialog;
- permanent delete/deactivate behavior.

Goal:

User can safely remove habits.

27. Initial Open Implementation Decisions

These are not product questions anymore, just later engineering decisions:

1. Whether to use `AlarmManager` directly for exact reminder times or `WorkManager` for more flexible background processing.
2. Whether deleted habits are hard-deleted or soft-deleted with `isActive = false`.
3. Whether uploaded images are copied into app-private storage or referenced through persistent URI permissions.
4. Exact brick grid dimensions for 30, 60, and 90 bricks.
5. Exact Compose animation approach for laying bricks.
6. Exact Room migration strategy during development.
7. Whether to keep task completion records forever or only current tier records for MVP.

My recommendation for development:

- soft-delete habits with `isActive = false`;
- use app-private image storage;
- keep business rules outside UI;

- define wall grid math before writing the renderer;
 - implement rollover processing before polishing visuals.
-

28. Summary Definition

Bricks is a local-first Android habit-building app where the user creates strict, fixed-goal habits and earns one brick only by completing every required task on a scheduled day. The wall grows from Bronze to Silver to Gold as the habit strengthens from Habit to Routine to LifeStyle. Misses have tier-specific consequences. The app uses visual progress, brick animation, and optional image reveal to motivate consistency while avoiding unnecessary productivity-app bloat.